
Final Project Report: Adversarial Co-Evolution of RL and VLM/LLM Agents in Multiplayer Games

Nima Kelidari¹
kelidari@usc.edu

Mahdi Salmani²
salmanis@usc.edu

Mohammadsaeed Haghi³
haghi@usc.edu

Abstract

Large Language Models (LLMs) demonstrate remarkable capabilities in common-sense reasoning, yet they often struggle with the stringent logical consistency and long-term planning required for complex games. Conversely, Reinforcement Learning (RL) agents excel at optimizing policies but suffer from data inefficiency and sparse rewards. This project investigates a hybrid framework to combine these paradigms in the game of Gin Rummy. We developed a comprehensive system including a Distributed Master-Worker Architecture for LLM inference, a Scientific Dashboard for real-time monitoring, and multiple training regimes: Curriculum-based PPO, DAgger (Imitation Learning), and Short-Term Reward shaping via LLM evaluators. While our Curriculum-Enhanced PPO agent achieved a 99.1% win rate against random baselines, our experiments revealed significant limitations in other approaches. Specifically, we demonstrate that DAgger fails to generalize effectively in this domain, Short-Term Reward shaping leads to myopic policy saturation ($L=5$ horizon), and LLM-in-the-loop training remains computationally infeasible due to high inference latency, even with distributed architecture. This report details the successful engineering of the training infrastructure and provides a critical analysis of why standard hybrid methods failed to outperform pure RL in this specific imperfect-information setting.

1 Introduction

The central problem this project addresses is the challenge of training robust, high-performance Reinforcement Learning (RL) agents for complex, imperfect-information games [1], utilizing Gin Rummy as our primary testbed [10].

1.1 The "Opponent Bottleneck" in RL

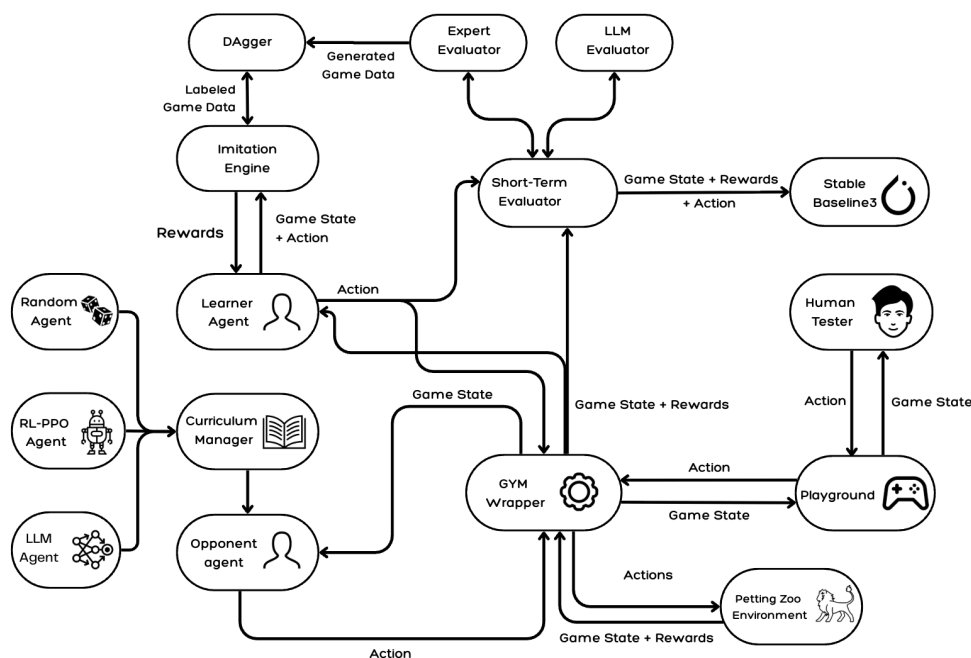
Training RL agents in competitive environments creates a "bootstrapping" dilemma. If the opponent model is a simple random-action agent, the trained policy often learns to exploit random errors rather than developing deep strategy. Alternatively, designing a strong, logical opponent (e.g., using MCTS [7]) is often unfeasible due to the lack of domain expertise or computational constraints. Utilizing human players is costly and unscalable. Large Language Models (LLMs) present a novel alternative: utilizing the "world knowledge" encoded in the LLM to act as a zero-shot opponent or evaluator [12, 3]. However, integrating massive models like Llama-3 [11] or GPT-4 into a tight RL training loop presents massive engineering challenges regarding latency and consistency.

1.2 Project Objectives

Our goal was to build a system that could:

1. **Distill Strategy:** Use an LLM or Expert to guide a lightweight PPO agent [6].

2. **Scale Inference:** Overcome LLM latency using distributed computing.
3. **Benchmark Regimes:** Rigorously compare Curriculum Learning, Imitation Learning (Dagger), and Reward Shaping.



To support our experiments, we engineered a robust infrastructure capable of handling hybrid AI workloads using the `stable-baselines3` library [4].

A major bottleneck in using LLMs for RL is inference time. A single game might require hundreds of LLM calls. To address this, we implemented a **Distributed Master-Worker Architecture**:

2.2 Scientific Web Dashboard

- **Unified Control:** The dashboard allows us to start/stop training, manage Master/Worker nodes, and visualize the game state in real-time.
- **Prompt Engineering:** We included a live "Prompt Editor" that allows us to modify the System Prompt injected into the LLM without restarting the server. This was crucial for our Prompt Comparison experiments.
- **On-the-fly Config:** Hyperparameters like Learning Rate, Batch Size, and Entropy Coefficients can be adjusted dynamically during a run to rescue stalling training sessions.

3 Methodology 1: Short-Term Reward System

Standard RL often suffers from sparse rewards the agent only knows if it won or lost at the very end of a long game [8]. To accelerate learning, we implemented a Short-Term Reward System that provides dense, per-step feedback, similar to concepts explored in intrinsic motivation [2].

3.1 Implementation Details

We modified the training loop to intercept every action and query an "Evaluator" before the environment processes the step.

- **Flag:** reward-system short⁴
- **Mechanism:** The system calculates a reward based on the alignment between the PPO agent's chosen action and the Evaluator's recommended action.
- **Shaping:** We implemented a shaping function: +1.0 for matching the evaluator, and −0.1 for a mismatch. This encourages the agent to mimic the evaluator's logic.

3.2 Evaluator Types

We supported two distinct evaluators:

1. **Expert Evaluator ('expert_agent.py'):** Uses a handcrafted heuristic based on 'hand_scoring.py'. It calculates deadwood and optimal melds to recommend the mathematically best move.
2. **LLM Evaluator ('llm_agent.py'):** Wraps the 'llm/player_handler.py' to query an Ollama model (default 'llama3.2:1b'). It includes fallback logic: if the LLM API times out or returns invalid JSON/text, the system defaults to a random action to prevent the training run from crashing.

4 Methodology 2: Imitation Learning (DAgger)

To further stabilize training, we implemented **Dataset Aggregation (DAgger)** [5]. Unlike pure Behavior Cloning, which trains only on expert trajectories, DAgger trains on the *student's* induced state distribution.

4.1 Training Loop ('dagger_train.py')

The DAgger process iterates through the following steps:

1. **Data Collection:** The current student policy π_s plays the game. At each step, the observation o_t is recorded.
2. **Expert Labeling:** We query the Expert Teacher π^* to label the optimal action a_t^* for that observation.
3. **Aggregation:** This pair (o_t, a_t^*) is added to a growing dataset $\mathcal{D}$.
4. **Supervised Training:** We train the student policy to minimize the cross-entropy loss between its output and the expert labels in $\mathcal{D}$.

We resolved significant compatibility issues by creating a shared 'agents/policy.py' class, allowing models trained via DAgger to be loaded seamlessly into the main PPO tournament loop.

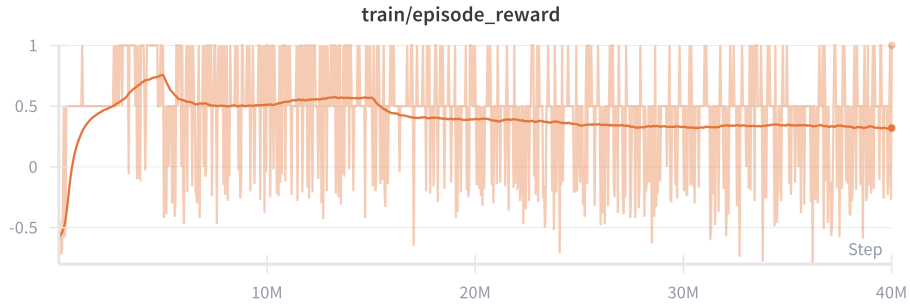
5 Experiments and Results

5.1 Phase 1: Curriculum-Enhanced PPO (The Success)

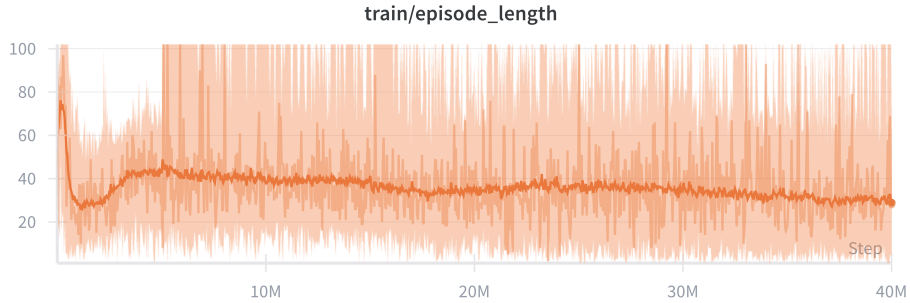
Our most successful approach was the Curriculum Learning PPO. Instead of complex rewards, we simply staged the difficulty of the opponents.

- **Stage 1 (0-5M steps):** vs. Random Opponent.
- **Stage 2 (5M-15M steps):** 50% vs. Random, 50% vs. Self (Frozen).
- **Stage 3 (15M-45M steps):** 20% vs. Random, 70% vs. Prior Models, 10% vs. Self.

The results were excellent. As seen in the figures below, the agent (Exp 5) achieved stable convergence and high win rates.

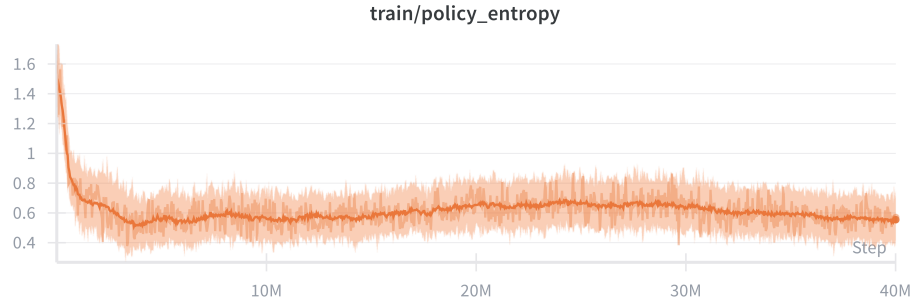


(a) Episode Rewards (Curriculum)



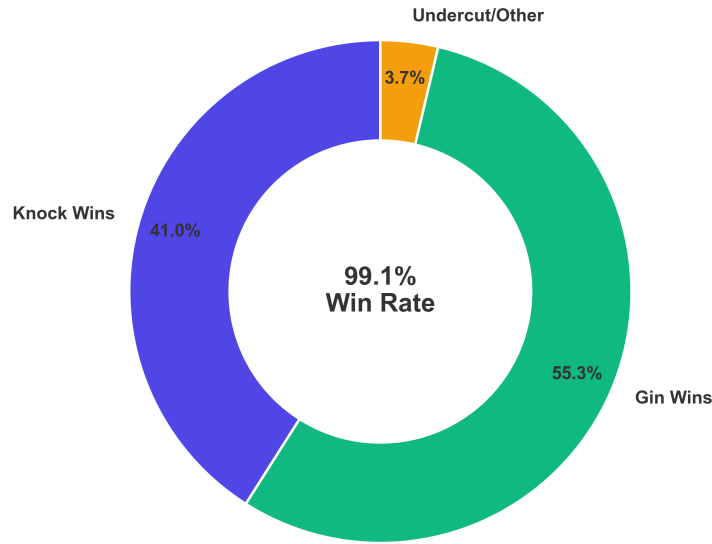
(b) Episode Length (Curriculum)

Figure 2: Curriculum PPO Results (Exp 5). The agent learns to extend episode length to maximize points (Gin vs Knock).



(a) Policy Entropy

Result: Balanced Strategy Distribution
(Curriculum Model)



(b) Win Rate vs Random

Figure 3: The PPO agent maintains healthy exploration (entropy) and achieves strong dominance over random baselines.

We also analyzed the effect of reward shaping (Gin vs Knock) in earlier experiments (Exp 3 and 4).

5.2 Phase 2: DAgger Failure Analysis

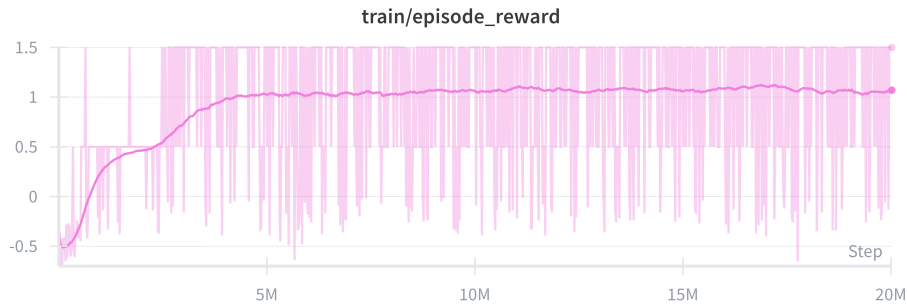
Despite the theoretical benefits of DAgger [5], our implementation failed to produce a competitive agent.

As shown above, the training process worked mathematically the loss went down. However, the **gameplay performance** was abysmal.

Conclusion on DAgger: The low training loss combined with poor performance suggests a severe **distribution shift** or "causal confusion." The student learned to mimic the expert's actions in specific states but failed to understand the underlying logic (e.g., tracking the opponent's hand), leading to catastrophic failure when it encountered states slightly outside the expert's trajectory.



(a) Exp 3: Knock-Biased Rewards



(b) Exp 4: Gin-Biased Rewards

Figure 4: Comparison of reward shaping. Exp 4 shows higher variance but higher potential rewards.

5.3 Phase 3: Short-Term Reward Failure Analysis

We hypothesized that Short-Term rewards would help. We tested Horizon Lengths of 2 and 5 steps.

5.3.1 Short Term Length 2

For a horizon length of 2, the model struggled to learn meaningful representations. The "lookahead" was too short to capture the strategic value of holding a card for a meld.

5.3.2 Short Term Length 5

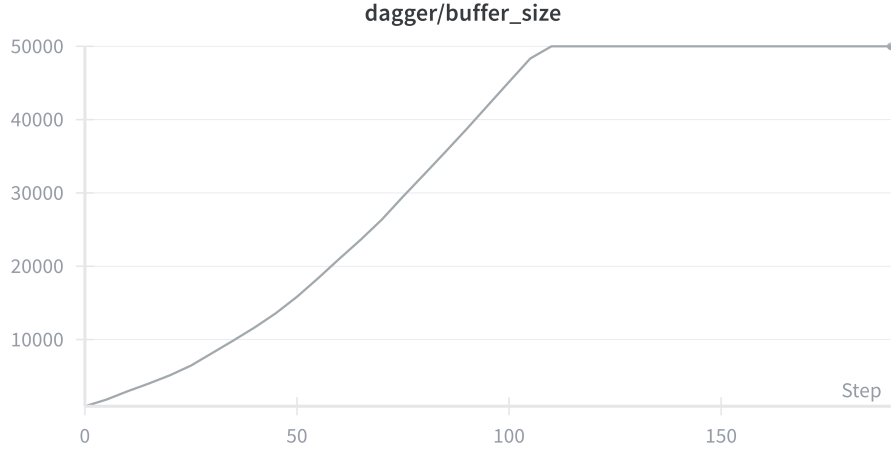
Increasing the length to 5 showed better signs in policy entropy (it did not collapse immediately). However, performance growth saturated after 500k steps. The agent learned to "farm" the short-term rewards without actually trying to win the game.

Conclusion on Short-Term Rewards: This approach proved ineffective. The dense rewards distracted the agent from the sparse, true objective (winning).

5.4 Phase 4: LLM Agent Performance

Finally, we analyzed the LLM agents themselves [12, 9]. While they are competent players, their utility as training partners is destroyed by latency.

Even with our **Distributed Master-Worker architecture**, the network overhead and intrinsic LLM generation time meant that training a PPO agent against an LLM would take weeks to reach the millions of steps required.



(a) Buffer Size Growth



(b) Training Loss

Figure 5: DAgger training metrics show successful optimization (loss decreases as buffer grows).

6 Discussion and Conclusion

This project conducted a comprehensive exploration of modern techniques for training agents in Gin Rummy. We successfully engineered a sophisticated infrastructure, including a distributed LLM system and a DAgger pipeline.

Our findings are conclusive:

1. **Curriculum Learning is King:** The most effective method remains the simplest: PPO with sparse (Long-Term) rewards [8], trained via a curriculum of increasingly difficult versions of itself. This produced our champion agent.
2. **Short-Term Rewards are Counter-Productive:** Dense rewards in this domain lead to ****myopia****. The agent optimizes for the immediate heuristic rather than the long-term win condition. We strongly recommend against using short-term shaping for Rummy-style games.
3. **DAgger does not Generalize:** While DAgger can clone behavior on the training distribution, it failed to produce a robust policy for the open-ended gameplay of Gin Rummy. The gap between "mimicking a move" and "understanding a strategy" remains too wide for simple supervised loss to bridge.
4. **LLMs are Tool Slow for RL:** Despite their strategic competence (60% win rate), LLMs are currently infeasible for online RL training due to latency. Even with a Master-Worker setup, the throughput is orders of magnitude lower than what is needed for sample-hungry algorithms like PPO.

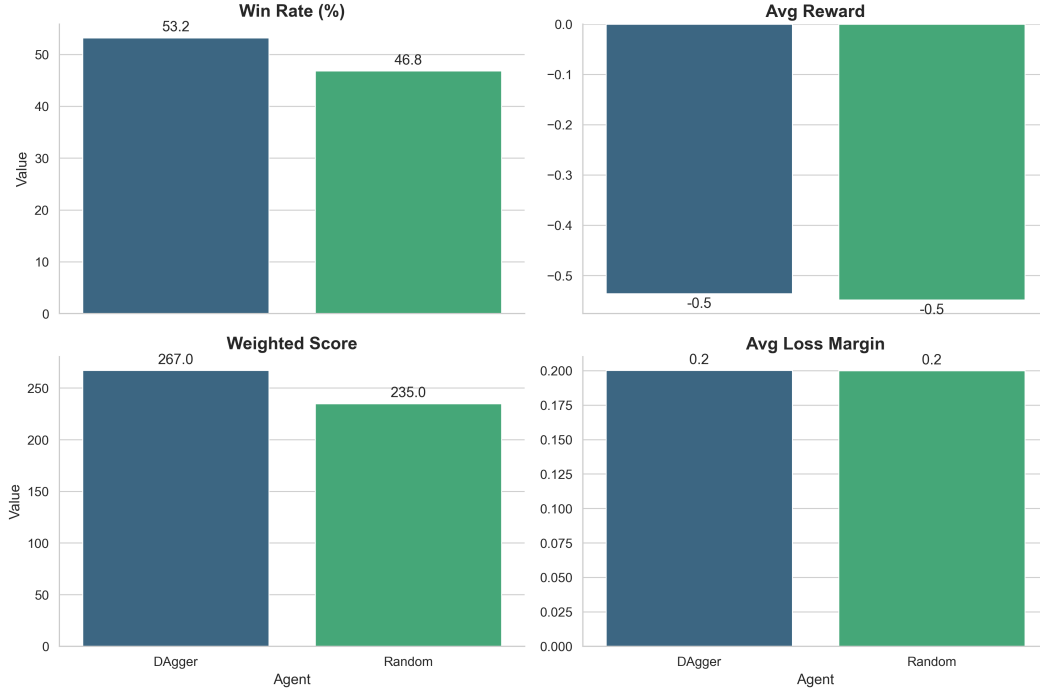


Figure 6: DAGger Performance Metrics. The win rate remains near zero.

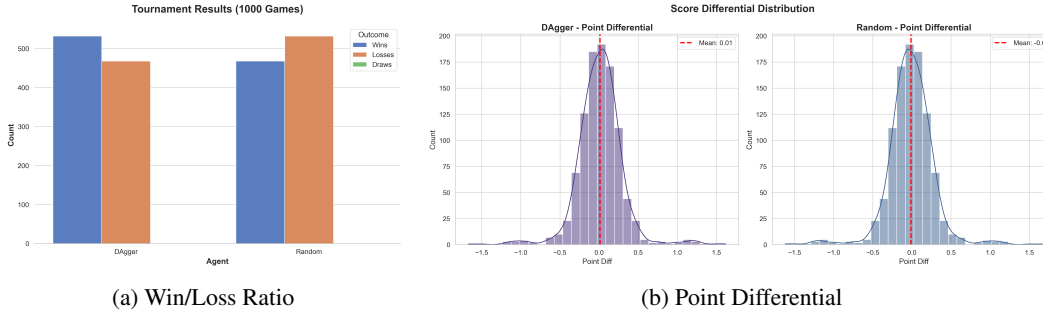


Figure 7: The DAGger agent consistently loses and maintains a negative point differential.

Future Work: The only viable path for utilizing LLMs is Offline RL. Instead of playing live, we should use the distributed system to generate a massive static dataset of LLM-vs-LLM games over several weeks, and then use Offline RL (e.g., Conservative Q-Learning) to distill that dataset into a policy, completely decoupling inference time from training time.

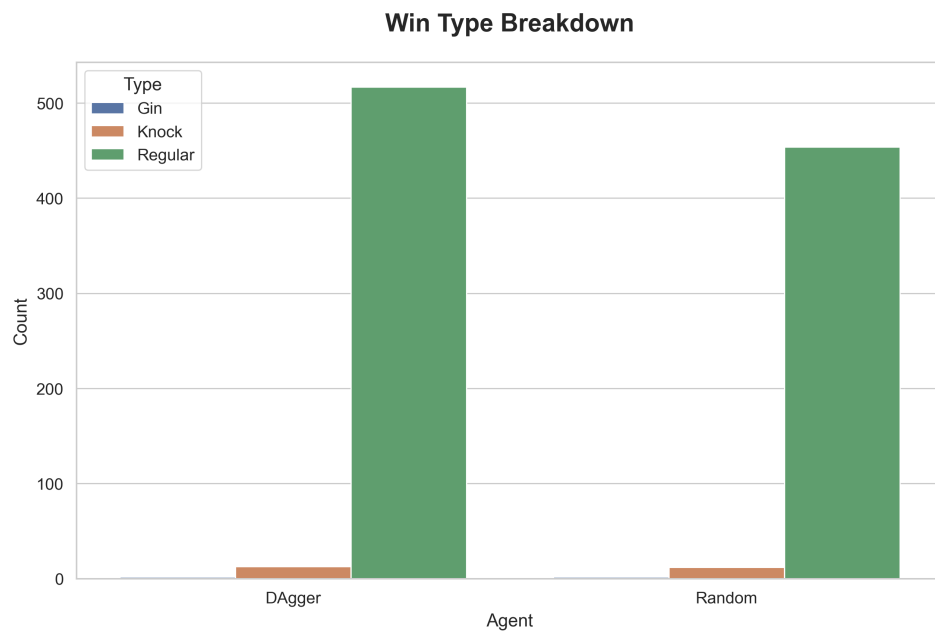
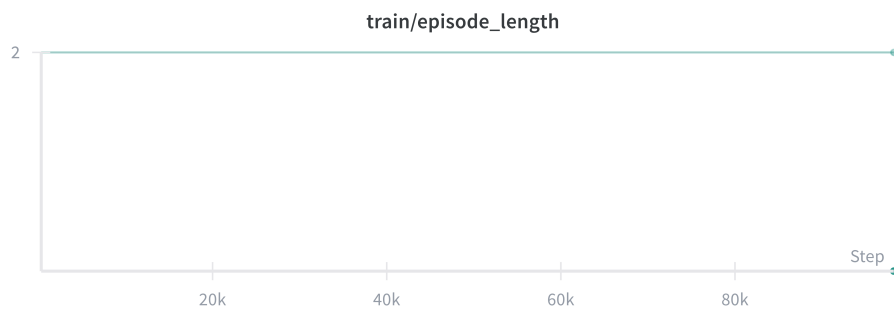


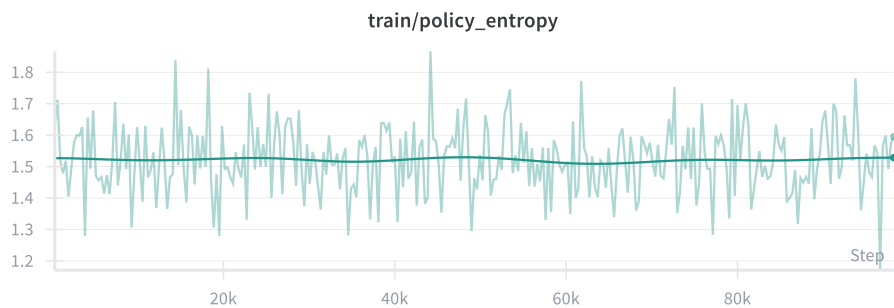
Figure 8: Win types for DAgger show almost no strategic wins.



(a) Episode Length ($L=2$)

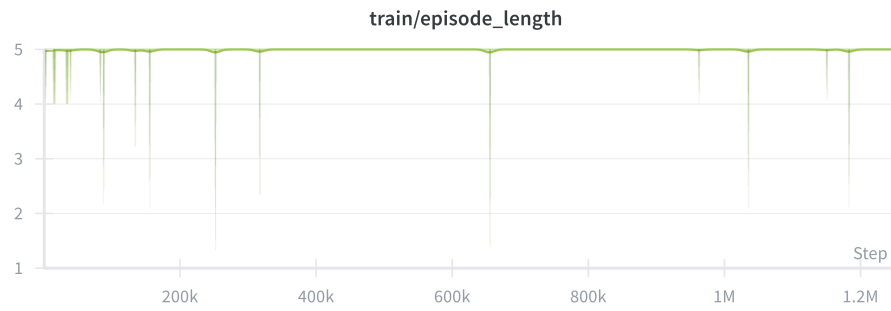


(b) Episode Reward ($L=2$)

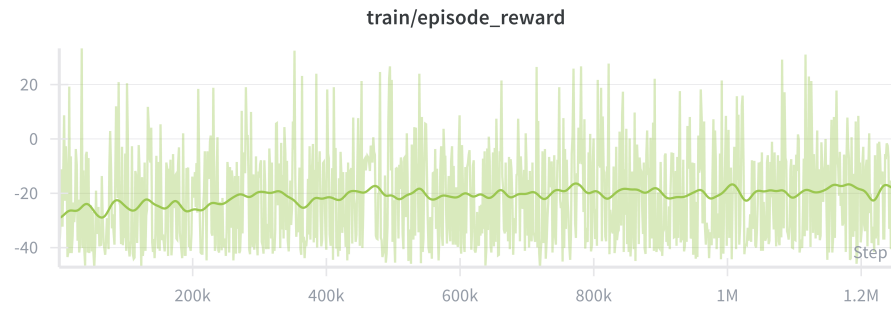


(c) Policy Entropy ($L=2$)

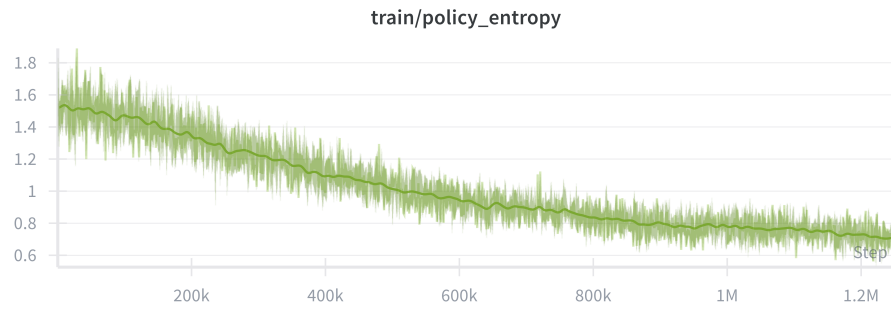
Figure 9: Short Term $L=2$: Flatling metrics indicate no learning.



(a) Episode Length (L=5)



(b) Episode Reward (L=5)



(c) Policy Entropy (L=5)

Figure 10: Short Term L=5: Entropy decreases, but rewards saturate early.

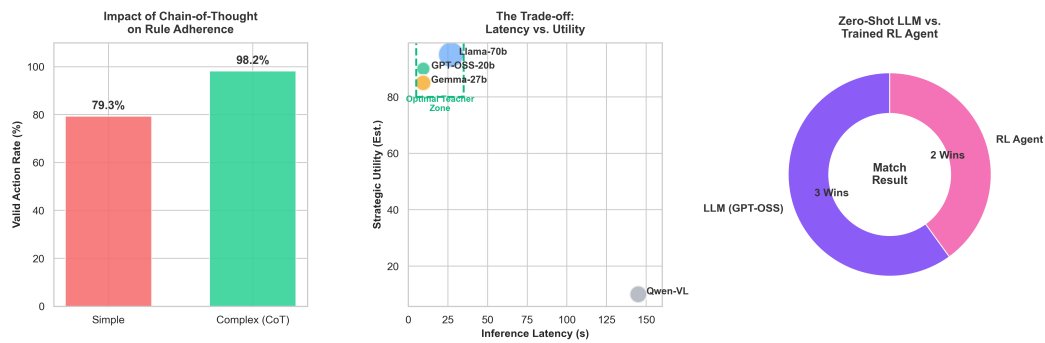


Figure 11: Left to right : a.Prompt Comparison: Complex prompts (Chain of Thought) are required for valid play | LLM Performance (Accuracy vs Response Time). Note the high latency | Win Rate Comparison: LLM (~60%) vs RL (~40%). The LLM is stronger but much slower.

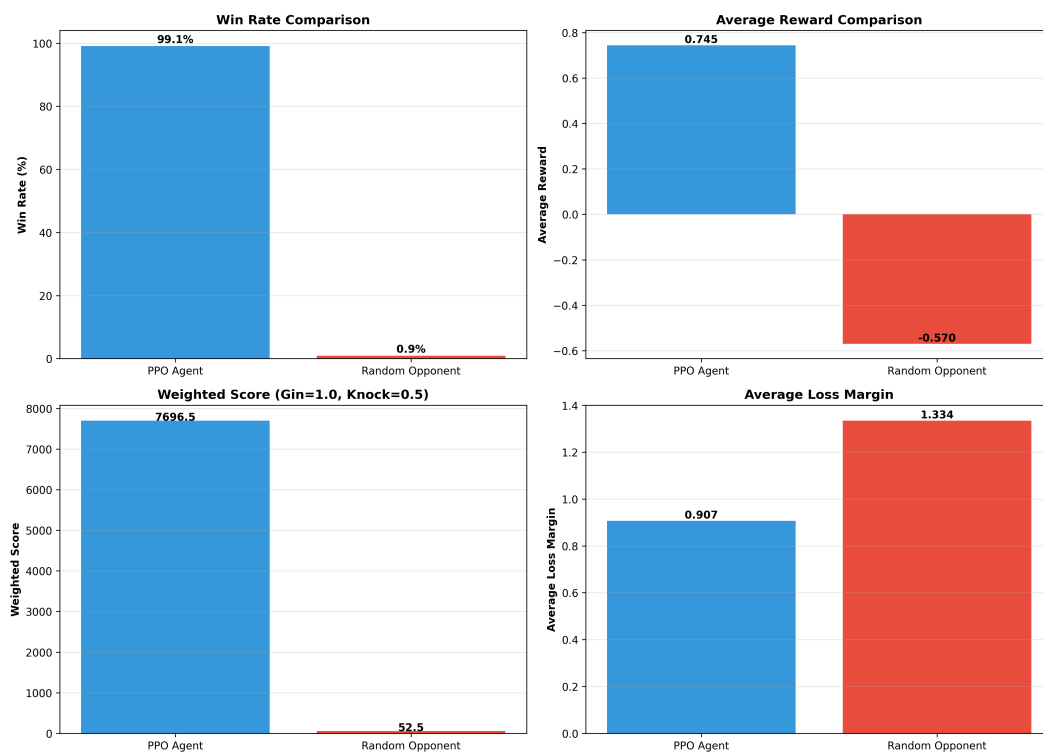


Figure 12: Win Rate Comparison: RL ($\sim 99.1\%$) vs Random Agent ($\sim 0.9\%$). The best model using curriculum learning with fascinating performance.

References

- [1] Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890, 2019.
- [2] Martin Klissarov, Pierluca D’Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, and Pascal andj Amy Zhang Vincent. Motif: Intrinsic motivation from artificial intelligence feedback. *arXiv preprint arXiv:2310.00166*, 2023.
- [3] Baolin Peng, Michel Galley, Pengcheng He, and Jianfeng Gao. Spag: Self-playing adversarial games for llm reasoning. *arXiv preprint arXiv:2402.13245*, 2024.
- [4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [5] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [7] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.
- [8] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [9] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Riviere, Mihir S Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [10] J Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, A Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34:15032–15043, 2021.
- [11] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [12] Mengjiao Yang, Yilun Wang, and Xinyu Feng. Large language models as agents in two-player games. *arXiv preprint arXiv:2302.09391*, 2023.

A Appendix: Configurations

A.1 Hyperparameters

- **PPO:** Learning Rate: $3e-4$, Batch Size: 2048, Gamma: 0.99.
- **DAGger:** Buffer Size: 100k, Epochs per Aggregation: 10.
- **LLM:** Llama-3.2-1B / 3B (Worker Nodes), Llama-3-70B (Enhancement Node).

A.2 System Configuration Files

```
1
2
3 # --- Training / PPO ---
4 training:
5   total_timesteps: 1000000
6   num_env: 100
7   turns_limit: 200
8   seed: 42
9   curriculum:
10    enabled: true
11    max_pool_size: 20
12    save_dir: "artifacts/models/ppo_gin_rummy/curriculum_pool"
13
14 # --- Evaluation / Short-term Rewards ---
15 evaluation:
16   reward_system: "long" # Choices: 'long', 'short'
17   type: "expert" # Choices: 'expert', 'llm', 'none'
18   game_length: 50 # Overrides turns_limit for short-term reward
19   episodes
20
21 # --- Distributed Architecture (LLM) ---
22 distributed:
23   master:
24     host: "0.0.0.0"
25     port: 8000
26     url: "http://localhost:8000" # Public URL for workers/clients
27     queues:
28       fast: "fast_queue"
29       slow: "slow_queue"
30
31   worker:
32     default_model: "llama-3b" # Default if not specified
33     polling_interval: 0.1 # Seconds
34     timeout: 300
35
36 # --- LLM Models ---
37 models:
38   llama-3b: "meta-llama/Llama-3.2-3B"
39   llama-70b: "meta-llama/Meta-Llama-3.1-70B"
40   qwen-32b: "Qwen/Qwen2.5-32B-Instruct"
41   llama-3.1-70b: "meta-llama/Llama-3.1-70B-Instruct"
42   gpt-oss-20b: "gpt-oss/20b"
43   gemma3-27b: "google/gemma-3-27b"
44   qwen3-32b: "Qwen/Qwen2.5-32B-Instruct"
45   default_evaluator: "llama-3b"
46   default_enhancer: "llama-70b"
47
48 # --- Client ---
49 llm_client:
50   timeout: 300
51   max_retries: 3
52   local_url: "http://localhost:11434"
```

Listing 1: Central YAML Configuration for the Distributed System and Agent Settings.

```

1 config = {
2     "algorithm": "PPO",
3     "policy": "MaskedGinRummyPolicy",
4     "total_timesteps": total_timesteps,
5     "learning_rate": learning_rate,
6     "n_steps": 512,
7     "batch_size": batch_size,
8     "n_epochs": 4,
9     "gamma": 0.99,
10    "gae_lambda": 0.95,
11    "clip_range": 0.2,
12    "ent_coef": ent_coef,
13    "vf_coef": 0.5,
14    "max_grad_norm": 0.5,
15    "randomize_position": True,
16    "weight_decay": 0.0,
17    "normalize_observations": True,
18    "reward_system": reward_system,
19    "evaluator": evaluator_type,
20 }

```

Listing 2: PPO Training Configuration Dictionary.

B Appendix: LLM Agent Prompts

B.1 Simple Prompt

```

1 You are "GinGenie," an expert Gin Rummy AI. Your goal is to win by
  forming melds.
2
3 **CURRENT GAME STATE:**
4 {game_state}
5
6 ---
7
8 **VALID ACTIONS:**
9 Here is the **ONLY** list of actions you can choose from:
10 {valid_actions}
11
12 ---
13
14 **CRITICAL WARNING! READ THIS!**
15 Look at the 'VALID ACTIONS' list.
16 * If you choose an action that is **NOT** on the list, you will fail.
17
18 ---
19
20 **YOUR FINAL RESPONSE**
21
22 First, complete your step-by-step thinking.
23 Then, on a **completely new line**, write **ONLY** the exact action
  string from the "VALID ACTIONS" list.
24
25 **Example 1 (Discarding):**
26 ... (my thinking process, Step 2 confirmed it's a DISCARD PHASE) ...
27 6. Final Choice: The 9 of Spades is my worst deadwood card (9 points).
28 discard 9 of Spades
29
30 **Example 2 (Drawing):**
31 ... (my thinking process, Step 2 confirmed it's a DRAW PHASE) ...
32 6. Final Choice: The discard pile card is useless.
33 draw from stock

```

Listing 3: The "Simple" prompt provided to LLM agents. This version often resulted in invalid moves.

B.2 Complex Prompt

```
1 You are "GinGenie," an expert Gin Rummy AI. Your goal is to win by
  forming melds.
2
3 **Your Goal (Melds):**
4 * **Set:** 3 or 4 cards of the *same rank* (e.g., 7 of Spades, 7 of
  Hearts, 7 of Diamonds).
5 * **Run:** 3 or more *consecutive* cards of the *same suit* (e.g., 5
  of Clubs, 6 of Clubs, 7 of Clubs, 8 of Clubs).
6
7 **Discarding Card Values (CRITICAL):**
8 * **J, Q, K:** 10 points (HIGHEST value)
9 * **10, 9, 8, 7, 6, 5, 4, 3, 2:** Face value (e.g., 9 is 9 points)
10 * **Ace:** 1 point (LOWEST value)
11
12 ---
13
14 **CURRENT GAME STATE:**
15 {game_state}
16
17 ---
18
19 **VALID ACTIONS:**
20 Here is the **ONLY** list of actions you can choose from:
21 {valid_actions}
22
23 ---
24
25 **CRITICAL WARNING! READ THIS!**
26 Look at the 'VALID ACTIONS' list.
27 * If you only see actions starting with 'discard...', this is a **
  DISCARD PHASE**. You have 11 cards and **MUST** discard one. You
  **CANNOT** draw.
28 * If you see 'draw from stock' or 'pick top card...', this is a **DRAW
  PHASE**. You **MUST** draw one.
29 * If you choose an action that is **NOT** on the list, you will fail.
30
31 **Mandatory Thinking Process:**
32 Follow this logic:
33
34 1. **Analyze Hand:**
35   * **Completed Melds (3+ cards):** [List any sets or runs. e.g., "
  Run: 5C, 6C, 7C"]
36   * **Potential Melds (2 cards):** [List all pairs and 2-card runs.
  e.g., "Set: 7H, 7S", "Run: 9D, 10D"]
37   * **Deadwood:** [List all leftover cards not in the above. e.g.,
  "2H, 5S, QK"]
38
39 2. **Critical Turn Check:** Look at the **"VALID ACTIONS"** list.
40   * What kind of actions are available? (e.g., "I only see 'discard
  ...' actions, so this is a DISCARD PHASE.")
41
42 3. **Check for Win (PRIORITY 1):** Is 'declare Gin' in the list?
43   * If YES: My decision is final. I will choose 'declare Gin'. (Go
  to Step 6)
44
45 4. **Check for Knock (PRIORITY 2):** If no Gin, is *any* action
  starting with 'knock with...' in the list?
```



```

46     * If YES: My decision is final. I will choose the first available
    'knock with...' action. (Go to Step 6)
47
48 5.  **Decide Meld-Making Action (based on Step 2):**
49     * **IF (from Step 2) this is a DRAW PHASE:**
50     * I must pick one card.
51     * Look at "Top of Discard Pile." Does this card help any of my **
    Potential Melds**?
52     * If YES: I will choose 'pick top card from discard pile...'.
53     * If NO: I will choose 'draw from stock'.
54     * **IF (from Step 2) this is a DISCARD PHASE:**
55     * I **MUST** discard. I cannot draw.
56     * I will look at my "Deadwood" list from Step 1 and find the card
    with the **HIGHEST point value** (e.g., J, Q, K, 10, 9...).
57     * I will choose the matching 'discard...' action from the 'VALID
    ACTIONS' list.
58     * **ELSE:** I will choose the first action in the 'VALID ACTIONS'
    list (e.g., 'declare dead hand').
59
60 6.  **Final Choice:** Review the **"VALID ACTIONS"** list one last
    time. State the *exact* string from that list that matches my
    decision from Step 5.
61
62 ---
63
64 **YOUR FINAL RESPONSE**
65
66 First, complete your step-by-step thinking (Steps 1-6).
67 Then, on a **completely new line**, write **ONLY** the exact action
    string from the "VALID ACTIONS" list.
68
69 **Example 1 (Discarding):**
70 ... (my thinking process, Step 2 confirmed it's a DISCARD PHASE) ...
71 6. Final Choice: The 9 of Spades is my worst deadwood card (9 points).
72 discard 9 of Spades
73
74 **Example 2 (Drawing):**
75 ... (my thinking process, Step 2 confirmed it's a DRAW PHASE) ...
76 6. Final Choice: The discard pile card is useless.
77 draw from stock

```

Listing 4: The "Complex" (Chain-of-Thought) prompt. This version was highly effective at ensuring 100% valid move generation.